

FRED TALKS

30th September 2026

CONTENTS

Surrender as a non-stupid life strategy

SASHA CHAPIN · 2104 WORDS

The End of Code Review? Or an Opportunity to Rethink it?

CHRISTIAN KÄSTNER · 2579 WORDS

Test Coverage Won't Save You

JENN COOPER · 1991 WORDS

GPT-6.1 Sol in Codex and ChatGPT Work

CODEX CHANGELOG · 63 WORDS

Surrender as a non-stupid life strategy

SASHA CHAPIN · 04 JUL 2026 · [SOURCE](#)

After getting everything I wanted in my 20s, and discovering that I was still miserable, I decided that I should no longer be in charge of choosing what I want, and I slowly settled into a frame of, “I’ll do whatever God seems to want from me.”

Some years after making that decision, I’m much happier. My life is richer, and I’m doing more for others. Regularly, I react with positive disbelief when I look at what my life has become. It looks like I’ve been executing a genius strategy. But in reality, I am living in a state of ongoing confusion, just seeing what happens.

If you’ve achieved a bunch of your goals, and you’re still not happy with your life, consider giving up on self-chosen plans. Instead, listen to your life, inner and outer, and see if you can feel a current. Then fall into it, especially if it has nothing to do with the story you had about where you were supposed to be going.

This choice is known as surrender. As with most cliches, I don’t think I can explain the profound experiential content of surrender—what it feels like, why it works. But perhaps I can lay out a constellation of propositions^[1] that will, together, illuminate a worldview very different from the one I once inhabited.

Knowing and not-knowing yourself

1.1 The planning part of your mind, the one that generates ambitions and hypothetical situations, is a small part of your overall intelligence. Anyone who is halfway good at sex, comedy, or any creative art will know this. Designing your life based on what that part of your mind wants is like designing your house to fit the needs of your right hand. Congratulations, your house is filled with fidget spinners and soap.

1.2 A large percentage of human suffering is anticipatory tension, or dreading a future experience such that you actualize its potential suffering in the now. Anticipatory tension is using the illusion of knowledge to generate the illusion of control. Much of what people call planning is this.

1.3 My best outfits are all selected by someone else. When I'm at a clothing store, and I meet an employee who seems engaged, I tell them to dress me for maximum attractiveness. They do a much better job than I do. Of course: I can't see myself as well as someone else can. This is why I try to listen to what the world apparently wants to do with me, rather than who I imagine I am.

1.4 Often, our self-image is a defense against the traits we reject in ourselves. "I love to be spontaneous," says the most calculating person you've ever met. Thus, if you're trying to live in self-loathing and denial, one way is to sculpt your life around the self-image you've decided on.

1.5 I've discovered that scrutinizing my motives too closely is a great way to stop creating. The unexamined life may not be worth living, but the fully examined life is a pinned butterfly in a glass case. I've concluded that my life is a grand artistic project, but I'm best off not knowing what it is.

1.6 You know that person who had important ripple effects in your life, based on a brief meeting or a kind gesture they extended to you, which meant nothing to them, but everything to you? Or that interesting stranger whose manner cast a brief spell on you, lingering in your memory disproportionately? That's you also, you're that person. Most of your impact on others, you will never fully apprehend or appreciate. Thus, it's not important to know what you're doing.

1.7 The proper end of introspection is not being filled with facts about oneself. Your introspection is successful if it is finite, if it leads you towards direct contact with experience. Get out of the way of your becoming.

Choice and authenticity

2.1 People feel most themselves when they are unselfconscious, which is to say, their actions emerge without apparent control. Think of the phrase "singing with abandon." In our most profound moments of intimacy, we have abandoned the wheel. Who is driving, at that moment? What if we simply let that driver have the wheel more often? It takes practice to notice when that's the person taking over the wheel—as opposed to our reactive, fear-based conditioning, which is also eager to take the job. Surrender is a skill.

2.2 When people talk about "being authentic," they are often referring to executing some program, some persona, that they've devised privately. Of course, our impression of such people is that they're unnatural and unoriginal. They assign us the job of reinforcing a character we never agreed on.

2.3 Surrender is something that happens through you, not something chosen^[2]. I couldn't have skipped the part of my life when I was contriving everything. And the way I'm living now wouldn't work without the skills I accumulated during that period of contrivance. If you're reading this and thinking, "gosh, I wish person x would see the wisdom of surrender," you have discovered an opportunity to surrender to interpersonal helplessness.

2.4 Often, the apparent struggle to make a decision is an attempt to meet an internal flailing quota, after which we will feel affirmed in the choice we already knew we were making. Sometimes you can just skip all of that, and admit that you're scared of what you know you will do. And then do it.

Goals, outcomes, effort

3.1 Discovered goals are entirely different from preconceived goals. Let's say you notice that when you play guitar and sing, people stop what they're doing to stare at you. The discovered goal of being a musician has presented itself. You are more likely to be successful than someone who privately dreams of being a musician without prompting from existence. Noticing such opportunities requires a lack of some previously decided agenda.

3.2 I've known many impressive people. Almost none of them started with a comprehensive vision of the grand work they would produce. Instead, they simply got into escalating amounts of trouble, and kept riding the trouble wave as far as it would go.

3.3 Excellence requires non-attachment to principles. All mediocre performers are the same: instead of improving on feedback from what happens, they react to what they think is supposed to be happening, or try to reinforce a theory they've internalized—perhaps an inaccurate approximation of someone they admire. This attempt at coherence is futile, since the mind of an expert makes no sense. It is filled with exceptions, contradictory principles butting up against each other. Perhaps the knowledge of how your life works is the same.

3.4 When a project in my life is working, it's typically marked by a sense of leverage. It's as if I've fallen into an updraft and it's propelling me. Meanwhile, when I'm privately excited about something, but there's no cooperation from the world, nothing comes together. The sensible conclusion is that my enthusiasm, alone, is not a helpful measurement.

3.5 It's nice to enjoy what you're doing. But there is a greater liberation available: accepting that the Work in front of you is your Work, and that has little to do with how you feel.

3.6 Another property of things that work out for me: they don't make sense to my rationalizing mind. There is the thought, "this isn't what I'm supposed to be doing," even as greater forces pull me forward. This is known as the refusal of the call to adventure. It's a routine part of adventure.

3.7 There is a criticism you could make of my advocacy of surrender. It is: "you're just lucky and narrativizing it." Yes. If you, too, have luck that seems unrelated to your conscious choices, consider letting that run the show.

Happiness

4.1 The norm for human life: gross injustice, corrupt rulers, disease and death, unequal fortunes, looming apocalypses. It's possible that a large portion of our species will die off—it's happened before, we've almost gone extinct previously, that's allowed in the game. If your basic needs are met but you cannot be content with the way it is now, that is not the fault of the way things are now. It's because you haven't surrendered to the basic properties of incarnating in human form.

4.2 Strangely, happiness isn't produced by getting what you want. Unhappiness is the product of the number of times per minute that you believe circumstances should be otherwise. "Enough" and "not enough" are mental stances that, very often, are uncorrelated with exterior facts. Perhaps this is one reason Jesus says it is "easier for a camel to go through the eye of a needle than for a rich man to enter the kingdom of God"—the more you focus on expanding your pile, the more ravenousness becomes a habit. By contrast, you can, right now, try believing that what's happening is what's supposed to be happening, in exactly the right order.

4.3 Satisfaction comes from giving yourself away. From realizing that you have been called to some occasion that requires everything from you, and saying, fine, fuck it, here I am, tear me apart, I will exhaust myself to set right this one irrelevant corner. It is possible to view your whole existence this way, as a process of giving away all, everything, of what was never yours.

4.4 Another profound source of satisfaction is fit: when you fit like a slender hand in the glove of your circumstances. Consider the possibility that this satisfaction is already trying to happen, but you are resisting it, in a doomed attempt to be happy. Settling into this fit is another way of describing what I mean by "doing what God seems to want."

4.5 I can't defend this intellectually, but it has been true in my experience: spontaneous compassion is the approach to ethics that feels best and works most often. If you want a plan for consistently being good, the dumbest one that will work, and therefore the most robust, is

“notice that prosocial behavior is nourishing, and that peace is only available if you’re on good terms with your conscience, and allow behavior to flow from that.” I’m not deluded enough to think this is always sufficient, but it’s a good start.

Love

5.1 I am capable of privately spoiling my wonderful marriage at any time, by noticing that it’s not the relationship I would’ve imagined being in. Meanwhile, a few months before meeting my wife, I wrote down a list of characteristics that the perfect partner would have, and I subsequently met that person. She was completely lovely—I find much to criticize in my own behavior from that period and nothing bad to say about her. And yet the relationship did not stick. How odd.

5.2 When I insist that a relationship must survive, I begin to resent it—I become burdened by all of the little moments of inauthenticity engaged in to shield the other person. The person becomes an obstacle, someone to navigate around. When I accept that any relationship could end, and surrender to the reality of people finding their own level, I become free to relate openly, and then actual closeness happens.

5.3 Panic is a part of love. I’ve learned that as I enter into any relationship that will alter me for the better, the manager in my head will come up with convincing reasons why it’s all wrong. This, too, is the refusal of the call; it is the fear of personal transformation masquerading as reason.

5.4 The older I get, the more that friendship is a discovered property. There are lots of wonderful people I meet who aren’t my friends. And then I meet someone and think: oh, that’s my friend, you’re coming with me. One of the best things you can do for yourself is to notice when you’re inexplicably bound to someone, and act on it without thinking.

5.5 We are drawn towards people because they are different. We are given the opportunity to love more when we find ourselves resenting them for the very difference that drew us to them. The opportunity is to release resentment and instead surrender to separateness. Ironically, this is how you feel less alone.

Thank you to Erin Shetron for editorial assistance. This essay was inspired a little bit by The Surrender Experiment, and a lot by [this totally wonderful meditation guide](#). Photo credit goes to Daido Moriyama.

This format is shamelessly stolen from [this fascinating essay](#) by “de Pony Sum.”

2 [\[find in text\]](#)

I first heard this point made by Adyashanti.

...

The End of Code Review? Or an Opportunity to Rethink it?

CHRISTIAN KÄSTNER · 26 SEP 2026 · [SOURCE](#)

Code review as we understand it today will probably not survive the shift to AI-generated code, but is that a problem? Code review is under pressure from agents producing code at an incredible pace, and reactions to that are quite split. There are many who want to protect code review as a core human software engineering practice, who put in place strict policies to preserve it. Others argue that code review is effectively dead, because it cannot keep up with the pace of production and will not survive competitive pressures if it becomes a bottleneck. Many experiment with automating code review at least partially. Implicit in this discussion is often the assumption that our prior code review practices were a gold standard, something to save or something to mourn. But is that really the case?

$$C = P + L \times (1 - E) + R$$

C Total cost (in \$/LOC)

P Production cost (in \$/LOC)
↳ Agents produce code faster

L Liability (in \$/LOC): Rework + Harm
↳ Lower quality, more harm and rework
↳ Higher quality, less harm and rework
↳ Agents lowering rework costs
↳ Cheaper substitutes: tests, proofs

E Effectiveness of Reviews (in %)
↳ Productivity pressure
↳ Deceptive plausibility
↳ Cognitive debt

R Review cost (in \$/LOC)
↳ Automated code review

A sweet spot for code review, historically?

Code review did not always look the way it does now (or at least the way it looked until coding agents took off). The contemporary practice that we now all know – tool-mediated, CI-integrated, lightweight review of individual commits, usually integrated with testing, static analysis, and other automation – is what reviewers call **modern code review** to distinguish it from other forms of code review and inspection.

Modern code review is called *modern* in contrast to older practices that many junior engineers have probably never seen and that we barely teach these days, most popular among them *Fagan-style inspections*. These were heavyweight processes, popularized in the 70s and widely studied through the 80s and early 90s: Multiple reviewers read small fragments of code line by line before release (not incrementally while code is created), often meeting to talk it through and sometimes reading it aloud. Fagan-style inspection was widely taught and written about (e.g., Wiegiers’ book “[Peer Reviews in Software](#)”). [Studies](#) showed it to be the most effective technique to find bugs we have, more effective than testing. But it is slow and expensive, easily costing more than 8 developer hours across 3 reviewers, a moderator, and the author for reviewing merely 400 lines of code. That cost limited how much code could be reviewed. Fagan-inspections were well known in principle but only sparingly used for high-stakes settings – often to review the critical core of a product.

The lightweight practice that eventually took over grew out of patch-by-patch review in open-source projects and converged, remarkably, on nearly the same practice in companies and in GitHub’s pull requests. [Studies](#) are clear that modern code review is pretty bad at catching bugs, catching mostly shallow ones and readability issues (though [better tooling](#) has given reviewers a chance to go deeper). However, compared to Fagan-style inspections, it is cheap enough to do for *all* changes. While modern code review is not great at catching bugs, it provides many other benefits: forcing “[developers to write code that other developers could understand](#),” fostering collective code ownership and awareness, supporting learning and mentorship, surfacing optimization opportunities, and enforcing conventions. Review has shifted, in [Rigby’s phrase](#), from a defect-finding activity to a group problem-solving one.

There may be many plausible reasons for the almost universal adoption of modern code reviews – whether companies copying successful open source practices (coined “inner source”) or just wanting to be more like Google, whether inertia, or whether developers just do not trust their interns – but let’s assume somewhat rational actors: If modern code review wouldn’t have provided any benefits, we would have stopped spending so much effort on it. For the sake of this argument, let’s assume, the community has eventually settled on an **equilibrium** where the value modern code review provides just justified its cost in a pre-AI world, whether through catching bugs or through those other benefits.

A simple model to untangle competing arguments

Before looking at how agentic coding puts code review under pressure, let's posit a simple model that will help to untangle various moving parts to help put various forces into perspective.

- **P**: The cost of producing code, in \$ per line.
- **L**: The liability of that code, in \$ per line, meaning the expected future cost of its quality problems. Let's assume liability has two parts: (a) **Rework**, the cost of finding and fixing bugs or in the worst case rearchitect the entire system. (b) **Harm**, the cost when problems escape to the outside world as security incidents, reputational damage, and losses to customers we may need to reimburse, sometimes after lawsuits. Rework is internal and is the main concern for low-stakes software, but harm is visible externally and can be substantial in high-stakes software.
- **R**: The cost of reviewing code, in \$ per line.
- **E**: The effectiveness of review at reducing liability, as a percentage.

Overall cost per line is then $P + L \times (1 - E) + R$. In a rational-actor world, teams invest in code review exactly up to the point where the last dollar spent on it just offsets the liability it removes.

For high-stakes code with high potential liability (L), it is rational to invest heavily (R) into making review more effective (E), up to Fagan-style inspections. For most code, stakes are not that high and liability comes mostly from rework. Also many other efforts have helped reduce liability without code review: Modern languages prevent whole classes of memory defects, static analysis catches many issues, and test generation and continuous-integration automation move testing into the cost of production. In that setting, shallow and fairly ineffective modern code review looks rational with modest cost for modest liability reduction.

Agentic coding shifts every part of the model, all at once

Agentic coding clearly has the potential to lower the cost of code production, but it can also influence the cost and effectiveness of code review, and the liability of typical code produced. The specific effects are often disputed, point in different directions, and interact with various things changing at the same time. The model can help us to untangle what we might expect to happen.

Production cost (P) is the obvious starting point but only part of the story. [↘ production cost] Agentic coding promises to drastically cut the cost of producing code (P). If nothing else changed (e.g., same code review, agentic code producing the same liability as human code), review cost (R) and liability cost (L) limit how much we can reduce overall cost. For example, even if production costs are reduced by 90%, overall costs would just be reduced by 30% if production accounted for only a third of the costs previously. This effect likely explains some of the discrepancies between frequent claims of 10X productivity improvement (which may even feel intuitively right seeing how quickly code is produced) and much lower productivity improvements measured in academic studies (rarely above 30%). If production cost is the only thing that changes, code review feels very much like a bottleneck limiting progress.

Agentic coding affects liability (L) in at least three plausible ways, and it is not obvious where we will land. When it comes to code quality and liability, the effect of coding agents is disputed.

[↗ liability due to lower code quality] First, there is some evidence (including our own study) that coding agents produce code of lower quality than humans, so that it needs more rework later. Also there are plenty of stories of harms from poor agentic code, like the multiple security incidents in the vibe-coded Moltbook and multiple AWS outages reported to be caused by agentic coding tools. All of that would point toward increased liability of code generated by coding agents, which could call for more scrutiny with code review.

[↘ liability due to better code quality] However, it is also plausible (even today with enough iteration and tool use, but plausibly more so with future improvements) that agentic code is of higher quality than human written code, lowering liability and the benefit of code review ($E \cdot L$).

[↘ liability due to cheaper rework] Second, just like code production, agents promise to make *rework* cheaper too, with automation in debugging and repair, which shrinks the rework part of liability. Especially in low-stakes settings, rework is likely the larger part of the liability cost, and lower code quality may simply not matter much if it can be fixed easily later.

[↘ liability due to cheaper review substitutes] Third, alternatives to code review get cheaper too: Tests, static analysis, and automated code review, and maybe one day even formal verification and proof-carrying code at scale, can catch problems early during code production, reducing liability before code review. If such activities become effective and cheap enough, they render code review less important for liability reduction.

I do not think it is clear yet how this will pan out. In low-stakes settings, self-reviewing agents, more testing, and cheap rework may seriously reduce most liability, but it seems less obvious how it will affect the kind of downstream harms that matter in high-stakes settings – security issues, safety issues, financial losses. When issues can create real harms, patching a system in production after harms have occurred may not be the best strategy.

Productivity pressure, surface plausibility of agentic code, and cognitive debt can each undermine review effectiveness (E). [↘ effectiveness due to productivity pressure] Many practitioners report enormous pressure to improve productivity with AI. With more code arriving from agents and review becoming the bottleneck, this translates into pressure to review faster, which can result in rubber-stamping, undermining any effectiveness modern code review may still have had.

[↘ effectiveness due to deceptive plausibility of AI generated code and cognitive debt] In addition, agentic code may be harder to review well, because it looks plausible and well written (automation bias), and because a reviewer who was not engaged in producing it often lacks the context to judge it (cognitive debt). Both issues lower review effectiveness, or would require more investment into code review to retain effectiveness.

These effects make preserving traditional human code review particularly challenging if most code is generated by AI. Insisting on manual review, but pushing more reviews through faster, likely just hollows it out. I fully agree that this will not be sustainable and code review will not survive in this form.

Automating the code review may lower review cost (R). [↘ review cost with review automation] Many of the quality problems that surface in modern code review, especially the shallow ones modern code review catches, can be found by machines mimicking code review. *Automated code review* by agents can arguably find many of those kinds of issues and lower review costs similar to how AI lowered production costs. Some offerings of automated code review can be quite pricey (e.g., Claude’s code review costs \$15-25 per reviewed PR), but they still likely undercut the costs of humans and they don’t share the human capacity bottleneck.

Lowered review costs from automated code review cannot be viewed in isolation. Whether automated code review is more or less effective (E) than humans, or effective only for particular classes of bugs, is still unsettled. Automation is likely able to go much deeper and be more systematic than a typical human review (closer to Fagan-style inspections with checklists), but it may also miss issues obvious to humans. If automation produces more false positives than human reviewers, it would push production cost (P) back up, though if production is cheap that may not matter much.

Indirect effects of code review, beyond finding bugs, may or may not matter, and they will decide the future of code review. Code review has many well understood benefits beyond its ability to find bugs, including learning and mentoring, (collective) code ownership, and general awareness – not captured in the simplified model (P, L, E, R). Pushing more code through the review process or automating code review away will obviously degrade those benefits too: If developers review less, they may gain less experience that helps them in producing code or being effective at those reviews still remaining. Developers may not understand their code base at all, making it harder for them to write good prompts to change it further, possibly increasing liability indirectly through cognitive debt. Or tooling can cover enough of what is lost, freeing up developers' time to focus on other parts that matter (e.g., requirements, specifications for test generation, infrastructure for testing in production).

Such effects are all plausible, but it is not clear to me how it will shake out. If we want to argue for code review, we should be clear about what benefits we want to gain from it (beyond finding bugs). And frankly, we should look for ways of getting those benefits without traditional code review. More research is needed, and in it we must be explicit about the mechanisms we expect and what effects we predict, whether on production cost, review cost, liability, or anything else.

Code review changed before

Heavyweight inspection never became the norm, because it cost too much to run everywhere, and the lighter practice that spread in its place exchanged rigor for scalability. I suspect code review will change again, or will be replaced by something looking completely different. The previous “modern code review” practices were probably close to an equilibrium, but agentic coding changes the forces in play, making that form of code review no longer sustainable.

Personally, I cannot predict where things will go. I think market pressures to increase productivity in an era of AI hype will make it difficult to preserve anything resembling our current modern code review practices when it becomes the bottleneck. But I also don't see modern code review as a practice to protect for its own sake, just because that's what we have previously done. I expect that coding agents will get much better at reviewing their own code during production, so that few of those more shallow and predictable issues remain that traditional code review or static analysis tools might have surfaced.

I think the bigger question is about all the other benefits we got from code review that were not about code quality, like coordination and mentoring. I think the better question to ask is how much those are still needed and how we can get them in forms that are not code review. For example, would better standup meetings or AI summaries of changes be able to replace the awareness benefits of code review? Would deliberate feedback and mentoring mechanisms

better support the junior to senior software engineering pipeline than code review? I think stepping back and rethinking a world without code review is a more useful exercise than tweaking existing code review practices.

Zooming out further, I think eventually the big question is really about where and how and how much humans will be in control in some form when producing software. Maybe that control is better focused on requirements, architectural design, test design, and security/safety safeguards, rather than reviewing lines of AI-generated code.

So my main point is to stop asking whether and how code review in its current form can be saved, but to have an open conversation about what we are trying to achieve here and the costs and benefits of alternative models.

Acknowledgment: This model grew out of a conversation with Courtney Miller about whether preserving code review is the right goal for researchers to support. I am no longer sure it is.

Code review is probably on its last legs, and we should focus on how to retain the indirect benefits it provided...

Test Coverage Won't Save You

JENN COOPER · 09 JUN 2026 · [SOURCE](#)

A year ago, if a codebase accumulated three different ways to do the same thing, somebody would usually notice.

A reviewer might leave a comment. A senior engineer would mutter something about "yet another helper," and eventually the team would clean it up. Maybe they'd write a short doc. People would learn, collectively, that this was not the way.

This was not a perfect system. Human review can be slow. People miss things. Pattern docs get stale. Engineers develop opinions that are 70% wisdom and 30% scar tissue.

Still, the friction did something useful. It kept codebases from drifting too quickly.

Coding agents change that.

A room with too much electricity.

"Now, this is a room with electricity. But it has too much electricity."

Once agents are writing a meaningful share of code, the question is no longer just "can we get working code faster?" Obviously, yes. We can. Sometimes hilariously so.

The more interesting question is: what does the codebase teach the next agent?

Because it does teach.

Every merged PR becomes a precedent. If the repository contains one clear pattern, the next agent has a decent shot at following it. If it contains three slightly different patterns, the next agent may extend one, combine two, or invent a fourth for reasons that are, in technical terms, vibes.

The funhouse builds itself

Coding agents, like some of us, have a chaotic inner goth teenager.

Their non-deterministic nature means they can be inconsistent, often in surprising ways. They can generate the weird little leap that solves the problem, or they can generate novelty where nobody asked for it.

The dangerous thing is that most of the code is fine, if you look at it in isolation.

A PR passes tests. The implementation makes sense. The file is clean enough. You look at the diff and think, yeah, sure, that seems okay.

But zoom out, and the codebase as a whole has started to get weird.

- three near-duplicate utilities
- multiple data-fetching patterns
- parallel abstractions solving almost the same problem

The codebase remains locally **cohesive**, while slowly losing global **coherence**.

That matters more than it used to, because even more than humans, agents are responsive to the signals around them. A clear codebase gives them clear precedent. A muddled codebase gives them confusion.

You can put principles in `CLAUDE.md` or docs, but they're... advisory. They compete with task instructions and inline comments for attention, get summarized when context windows compact, and depend on the agent attending to the right instruction at the right moment.

If your codebase contains three slightly different ways to solve a problem, the next agent has to infer which one is canonical.

From the agent's point of view, this is not irrational. The repository gave it mixed evidence.

The scary outcome is not code that obviously fails. It is code that superficially keeps working, while the shape of the system increasingly gets worse.

Tests are nice

Most discussions about AI-native development jump from this problem – agents' tendency to accumulate tech debt – directly to tests.

And yes, across the industry, teams are writing dramatically more tests than they ever have. Agents have made high test coverage affordable in a way it never used to be.

Recently, [Garry Tan argued](#) that the primary way to keep AI agents on track is 90% test coverage:

| *Tests are the ratchet. 90% coverage, every PR, no exceptions.*

And indeed, test coverage is the simplest kind of ratchet: a mechanism that allows motion in one direction only, like a socket wrench that turns the bolt forward but never lets it spin back. Once a test locks in a behaviour, it becomes difficult to accidentally regress.

But it's worth being specific about what tests actually do.

Unit tests check that a function still returns what it returned before. Integration tests check that pieces still wire together the same way. E2E tests reach further: they check whether the product still does what it used to, and the assertions you prioritize there should be an important act of human judgment.

But they all share the same basic shape: tests verify that code does what it did before.

Whether what it did was even the *right way to do it* is a separate question.

Tests ratchet behavioural sameness.

Meanwhile docs and evals can pin down reasoning and behaviour bars, which is sometimes more important. But none of these is directly looking at the *shape* of the system.

90% coverage of good patterns

A coverage ratchet only improves the codebase if the patterns it's locking in are already good ones.

If a questionable pattern already exists in the code, the next agent is more likely to extend it. The tests generated around that implementation only reinforce it further.

Over time, high coverage can unintentionally fossilize architectural drift, rather than prevent it in the first place.

At first, especially during prototyping, this can be easy to miss, because nothing looks obviously broken. Coverage stays high. The system works. But over time, certain parts of the codebase become strangely resistant to cleanup.

Tests that protect the wrong behaviour create drag against coherence. The harder you push for coverage, the more deeply you can lock in the shape you happened to start with.

This isn't a criticism of tests. Tests are necessary. But tests primarily validate behaviour, while many of the emerging failure modes in agent-generated systems, especially as they grow, are failures of *shape*.

A load-bearing poster.

"Uh, I wouldn't take it down if I were you. It's a load-bearing poster."

Fitness functions protect the shape of the system

Unit tests ask whether the code still does what it did before. Fitness functions ask a different question: is the system still shaped the way we want?

Most codebases already have a few simple versions of this. Linters, type checks, and import-boundary rules all encode some idea of acceptable shape, at the micro level.

But in agentially-maintained codebases, we need an additional kind of macro fitness function. It has a couple of important jobs:

- catch bad agent choices at the moment of work
- redirect agents with the right amount of friction for the violation

At Forestwalk, for example, we have a fitness check that prevents tool handlers from calling back into our AI services. Tool handlers run inside the agent's own loop, so any handler that kicks off more inference can stack model calls inside model calls and freeze a live meeting. Agents sometimes accidentally introduce this pattern, so our fitness check flags it and ensures it's not committed.

Other fitness tests are ratchets that apply pressure to measurable properties – heuristics for shape like file size, bundle growth, prompt size, import fanout, and dependency count.

For these, the ceiling starts at the current value (e.g. 837kb) and flags to agents and humans if a PR would unduly increase it. For instance, when an agent adds an unnecessary new dependency that blows up our frontend bundle, the check fails.

The important part isn't just the limit itself. It's the “just in time” agent coaching.

At first, we found agents would sometimes over-optimize against whatever metric the checks exposed, with great resourcefulness and in ways we did not want. But refining the flags to give common sense guidance helped steer the agents into considering tradeoffs correctly.

For example, an agent might see this just in time:

```
❌❌❌❌❌❌ Failed Checks: 1 ❌❌❌❌❌❌
```

```
FAIL src/frontendBundleBudget.test.ts > Frontend bundle size budget
AssertionError: Frontend bundle grew +29 KiB (700 -> 729 KiB; fails a
```

This ratchet is review pressure, not a mandate to shrink the number a

Diagnose first:

- Is the growth accidental (dependency, duplicate/dead code, broad im
- Does the added JS unlock user-visible value?
- Can cleanup reduce it without hurting UX, accessibility, tests, or

Quality-preserving fixes:

- remove duplicated rendering paths or dead production code
 - move repeated styling or static configuration out of lazy-loaded JavaScript
 - lazy-load rarely used flows instead of loading them on the main path
- (more examples here)

Do not fix this by:

- omitting requested scope or deferring needed UI work
 - removing aria-label/title text or visible affordances
 - deleting helpful empty/error-state copy
- (more examples here)

For intentional growth, (procedure here) and explain why in the PR.

If you simply tell agents to "reduce bundle size," they will sometimes do naive things like:

- delete empty-state copy
- replace styled controls with browser-native ones
- inline dependency code to avoid import costs
- remove accessibility affordances

Instead, the idea is to not just explain what a given fitness test is protecting, but also explain what is *not allowed* to be sacrificed in the process, and do so at the moment the agent is working.

In contrast, the same guidance in `CLAUDE.md` is often skimmed past or applied inconsistently by agents. A just-in-time check, on the other hand, creates desired friction between the agent and a finished task. Where humans get tired and occasionally creative about routing around friction like this, agents are quite willing to schlep through feedback and loop until the check is passing.

A master bedroom.

"Come on in, it's your master bedroom!"

What the checks can't see

Fitness functions are good at catching things precise enough to encode.

That is useful, but coherence is not always that crisp. Some drift is obvious only in relation to the rest of the codebase: this new thing is too close to an existing thing, this test is protecting a pattern we no longer want, this change is making the next change harder.

So in addition, we use a few non-deterministic checks on top of the deterministic ones.

One runs before a PR is opened. It scans the proposed change against the existing codebase, and asks whether the agent has introduced or expanded a competing pattern. When it finds one, it points to the existing pattern, explains why the new one looks redundant, and asks the agent to either extend the original or justify why this case is genuinely different.

Most of the time, the agent course-corrects quickly.

We run a similar check again at the PR review level, where it has more context and can catch subtler pattern drift.

Another check runs nightly and looks for tests that have fossilized patterns we have since decided against. Its mission is to find tests that protect old implementation choices that, if left alone, would teach future agents to preserve patterns we no longer want.

These checks are not magic. They miss things. They can be noisy. They still need human judgment around them.

But the goal is not to eliminate judgment. The goal is to move more human judgment into places where it can be highest leverage.

Making coherence the easy path

Fitness ratchets are not the whole answer to maintaining coherence in agent-generated codebases.

Fitness functions are one layer. Pattern drift checks are another. Product verification, code review, evals, documentation, repository structure, and the actual taste of the humans steering the system all matter too.

However, it's worth investing regularly in the early layers: the ones closest to the moment an agent is making an implementation choice.

That is where we have found these types of fitness functions useful. They do not solve coherence by themselves, but they add friction in the right places. Bad patterns get interrupted earlier. Good patterns have a better chance of becoming precedent.

The same feedback loop that spreads bad patterns can be used to spread good ones.

So there's hope. When you see this pattern work, it soothes some of the worry we all have about agents causing chaos. They're a lot more useful when the codebase gives them an obvious next move. The right pattern is easier to find. The weird fourth version of the thing has a harder time looking reasonable.

It's not perfect. Keeping a growing codebase coherent still requires human judgment, architectural intuition, and social coordination. The system still needs to be built.

But that system, itself, increasingly feels like the work.

If the codebase trains the next agent, then the job is not just to write better code.

It is to build the environment where better code is the obvious thing to write.

GPT-6.1 Sol in Codex and ChatGPT Work

CODEX CHANGELOG · 29 SEP 2026 · [SOURCE](#)

GPT-6.1 Sol in Codex and ChatGPT Work

GPT-6.1 Sol offers near-Astra performance for complex work at a lower cost than Astra. Consider it for repeated, long-running work across code, apps, and documents. Availability depends on your plan, client, and workspace settings.

Use `gpt-6.1-sol`. See [Models](/codex/models#gpt-61-sol) for availability and [model selection](/api/docs/guides/model-selection#when-to-consider-gpt-61-sol) to compare models.